Listing 5.40 Deleting documents from three different movie indexes

```
POST old_movies,classics,movie_reviews/_delete_by_query
{
  "query": {
    "match_all": {}
  }
}
```

Again, be careful about issuing `delete` queries, because you may lose the whole data set! Unless you wish to purge a complete data set, exercise delete operations with caution in production.

We have been indexing the documents individually so far, but in the real world, there's always a case to index large sets of documents simultaneously. We may have 100,000 movies to read from a CSV file, or 500,000 currency foreign exchange rates fetched from a third-party service to be indexed into our engine. While we can usually use an ETL (extract-transform-load) tool like Logstash to extract, enrich, and publish data; Elasticsearch provides a bulk (`_bulk`) API to index messages in bulk. We discuss the bulk API in the next section.

5.6 Working with documents in bulk

So far, we've indexed documents individually using Kibana. Indexing a single document or handful of documents using the API methods is straightforward. This is well and good for development purposes, but we rarely do this in production. It is cumbersome and error-prone for a larger data set (for example, when extracting a larger number of records from a database).

Fortunately, Elasticsearch provides a `_bulk` API to index large quantities of data to index large data sets simultaneously. We can also use the `_bulk` API to manipulate documents, including deleting them.

The `_bulk` API accepts a `POST` request, which can perform `index`, `create`, `delete`, and `update` actions simultaneously. This saves bandwidth by avoiding multiple trips to the server. There is a special format for the `_bulk` API, which you may find a bit weird but is not difficult to get your head around. Let's look at the format first.

5.6.1 Format of the _bulk API

The `_bulk` API consists of a specific syntax with a `POST` method invoking the API call (see figure 5.15). The request body consists of two lines for every document that needs to be stored. The first indicates one of the actions to be performed on the document: `index`, `create`, `delete`, or `update`. The document is described in the second line, which we cover shortly.

Once we choose the action, we need to provide a value for this action key with metadata. The metadata is usually the name of the document's index and the document ID. For example, the metadata for a document with ID 100 in a `movies` index is `"_index":"movies","_id":"100"`.